Week 3 — Level 2: Intermediate

Lists: Collections of Things

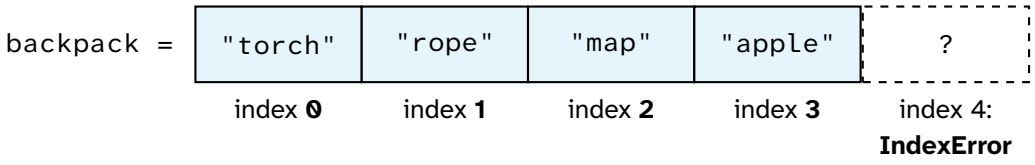
A list keeps many values in order, and the first one is number 0.

What you will learn

- How to keep many values in one **list**.
- How to find an item by its **index**, starting from 0.
- How to add items with `append` and count them with `len`.
- How to visit every item with a `for` loop.
- How to pick colors that work for everyone.

The big idea: a row of lockers

A list is like a row of lockers. Each locker has a number painted on it, called its **index**. The first locker is number **0**.



Making and using a list

```
backpack = ["torch", "rope", "map"]
backpack.append("apple")    # adds to the END of the list
print(len(backpack))        # 4   how many items
print(backpack[0])          # torch   the first item
print(backpack[-1])         # apple   -1 means the last item
```

With 4 items, the index numbers are 0, 1, 2 and 3. Asking for `backpack[4]` crashes with `IndexError: list index out of range`. This is called an **off-by-one** bug.

Two ways to loop over a list

Visit every item	Visit every index number
`for item in backpack:` `    print(item)`	`for i in range(len(backpack)):` `    print(i, backpack[i])`
Use it when you only need the items.	Use it when you also need the position number.

Colors for everyone

Some people see colors differently, often red and green. These eight **Okabe-Ito** colors stay different for almost everyone. Use them, and always add a label, number or shape too, so color is never the only clue.

orange	`#E69F00`		blue	`#0072B2`
sky blue	`#56B4E9`		vermillion	`#D55E00`
bluish green	`#009E73`		reddish purple	`#CC79A7`
yellow	`#F0E442`		black	`#000000`

Exercise 3.1: Palette Stripes

Folder: `exercises/week03/ex-palette-stripes`. Press **Run**: only one stripe appears.

1. TODO 1: add blue and vermillion with `palette.append(...)`.
2. TODO 3: draw a stripe for every color with `for i in range(len(colors)):`. Label each stripe `str(i)` and move `x` along.
3. TODO 4: print the first color (`palette[0]`) and the last color (`palette[-1]`).
4. My palette has _____ colors, so the labels go from `0` to _____.

Exercise 3.2: Adventure Inventory

Folder: `exercises/week03/ex-inventory`. Press **Run**. It crashes!

1. Read the error from the bottom up. Error name: _____ In which function? _____
2. Draw the backpack `["torch", "rope", "map", "apple"]` as boxes with their index numbers:

3. TODO 3: fix `last_item()`. TODO 1: append in `add_item()`. TODO 2: loop in `has_item()`.
4. Check your work with `check_my_work.py`.

Exercise 3.3: Top 3 High Scores

Folder: `exercises/week03/ex-high-scores`. Two new tricks: `sorted(scores, reverse=True)` makes a new list, biggest first, and `best[:3]` takes the first three items.

1. Predict: the top 3 of `[120, 45, 300, 80]` are _____
2. TODO 1: sort, then take three. TODO 2: print `1st: 300`, `2nd: 120`, `3rd: 80` using the `places` list.

Vocabulary box

list Many values kept in order inside square brackets, like `["torch", "rope"]`.

item One value inside a list.

index The position number of an item. The first item has index `0`.

append Add an item to the end of a list: `backpack.append("map")`.

len Counts the items in a list: `len(backpack)`.

IndexError The error you get when you ask for an index that does not exist.

off-by-one A bug where a number is one too big or one too small, like using `len` as the last index.

accessible Designed so that everyone can use it, including people who see colors differently.

Reflection

With a list of 10 items, what is the index of the last item? Why?

Describe a list you could use in a game you would like to make:

Week 3 Level 2: Intermediate

Lists: Collections of Things

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Index practice (paper, 3 minutes)

```
1 team = ["Ada", "Kai", "Mei", "Omar", "Zoe"]
```

`team[0]` = _____ `team[3]` = _____

`team[-1]` = _____ `len(team)` = _____

Does `team[5]` work? Why or why not?

★ Core (everyone): Loot Bag (Thonny, 10 minutes)

Open `exercises/week03/hw-loot-bag/starter.py`. You open a treasure chest several times and keep the loot in a list.

1. Press **Run**. The bag is empty!
2. TODO 1: in `fill_bag()`, repeat `how_many` times and append `open_chest(chest)` to the bag.
3. TODO 2: in `count_item()`, add 1 to `count` each time a thing in the bag matches the item.
4. Run it three times. Is the loot the same each time? Why not? _____
5. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Rare treasure

`"gold coin"` is in CHEST three times, so it is three times as likely as the ruby. Add a super-rare item that appears only once, and make `"old boot"` more common. Then open the chest 20 times: how many of your rare item did you get? _____

Creative (make it yours): Design a palette

Choose 4 colors from the Okabe-Ito list for a game or poster you would like to make. Write the list as Python, then sketch your design. Add a label or shape to every colored part, so someone who sees colors differently can still understand it.

Self-check: tick each box when it is true

- ☐ I know the first item of a list has index `0`, and the last has index `len - 1` (or `-1`).
- ☐ My `fill_bag()` uses `append` inside a loop.
- ☐ `check_my_work.py` shows `[PASS]` for my loot bag.
- ☐ My palette design uses labels or shapes, not only colors.